Implementing Explicit Congestion Notification (ECN) in TCP for IPv6

Chris Chen, Hariharan Krishnan, Steven Leung, Nelson Tang
{chenc, hari, stevenl, tang}@cs.ucla.edu

Computer Science 219

Prof. Lixia Zhang

December 4, 1997

Abstract

This paper discusses our design and implementation of Explicit Congestion Notification (ECN) for TCP on IPv6. The first part of the paper describes how ECN marks packets instead of dropping them to signal incipient congestion. Marking packets instead of dropping them carries a number of benefits, such as avoiding costly delays waiting for retransmissions timeouts or dropping packets on flows sensitive to even a single packet loss. The next part of the paper then reviews what changes are required to routers and to hosts to implement an ECN scheme. Our specific implementation is described, including justifications for some of our design decisions. We then explain how we tested our implementation, evaluating both only the router's behavior and the end-to-end performance of our system. Finally, we analyze our testing results and offer conclusions and other lessons learned from our research.

1. Introduction

Over its twenty-year history, researchers have extensively developed TCP [P81] to provide the best service as a reliable transport protocol. Part of the protocol's development focused on its ability to dynamically adapt to network congestion. Algorithms such as slow-start [J88] adjust TCP's transmission rate to achieve effective throughput even in adverse conditions. However, most of these algorithms probe for congestion by sending more and more packets until one of them is dropped. Dropping packets can be very undesirable for a number of reasons. For example, dropping all incoming packets at one router can result in the global synchronization [FJ94] of neighboring routers and hosts, which further degrade the efficiency of the network.

What the Explicit Congestion Notification (ECN) algorithm [FJ93] attempts to do is eliminate the time wasted waiting for a timeout by explicitly signaling when congestion is occurring. ECN does this by using a bit in the IP header which notifies the packet's recipient that the packet encountered congestion somewhere along the path from the sender. It is then up to the receiver to inform the sender that congestion is occurring and that the sender should slow down.

Ramakrishnan and Floyd have proposed to add ECN to TCP for IPv6 [RF97]. In conjunction with Random Early Detection (RED) [FJ93], a router queue-management scheme, ECN provides all the benefits of congestion detection and avoidance with less of the costly overhead of packet drops. We have implemented ECN for TCP and IPv6 on FreeBSD 2.2.2 systems.

In Section 2 of this paper, we review how ECN works and what changes are required to implement ECN. Section 3 describes how we tested and verified our implementation. Results of our tests are presented in Section 4, and our analysis of these results is provided in Section 5. Finally, in Section 6 we give our conclusions and talk about areas of future research.

2. Explicit Congestion Notification

The RED algorithm already has a notion of "marking" packets to signal congestion. However, the current design of RED only knows to drop marked packets in order to provide congestion feedback to the sender. As previously mentioned, dropping packets are nearly always a less-than-optimal signal of congestion. We would rather treat marked packets in a way which delivers them yet still conveys congestion.

This is exactly what ECN provides. With ECN, a marked packet simply has a bit turned on in its header to signal congestion. The packet is allowed to arrive at its destination. The receiver then signals back to the sender that congestion was encountered in the path that the original packet

traversed. Some benefits of ECN are avoiding the delay of slow-starting when congestion is signaled, avoiding global synchronization, reducing packet delay due to router queue length management, and avoiding packet drops, which can harm flows which are sensitive to packet loss.

In our implementation, we have chosen a bit in the Flow Label field of the IPv6 header [DH95] to indicate congestion. This bit is called the **ECN-Notify bit**. We also need a way for routers to know if a packet's sender and receiver are both using the ECN algorithm. This requires a second bit, also in the Flow Label field, called the **ECN-Capable bit**. A sender using ECN will always set the ECN-Capable bit for its outgoing data packets, giving the router permission to mark packets using the ECN-Notify bit. Our revised IPv6 header thus looks like Figure 1, where **EC** is the ECN-Capable bit and **EN** is the ECN-Notify bit.

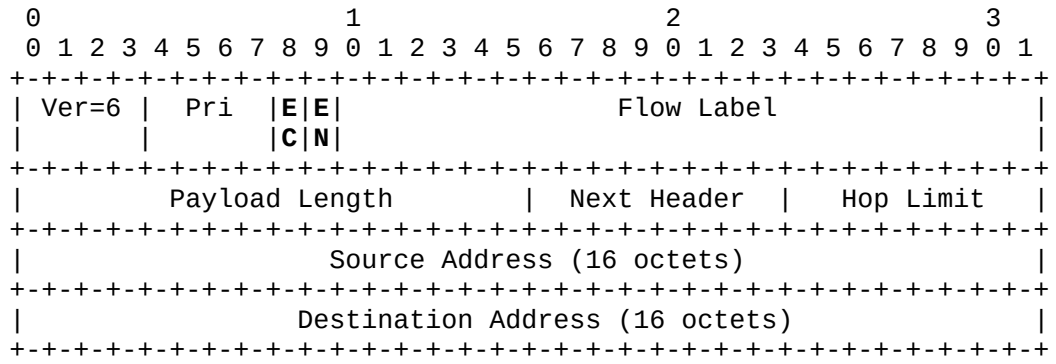

Figure 1: Modified IPv6 header format

The Flow Label field seemed to be the most appropriate field from which to take our ECN bits because it had the least specifications. The Priority field was considered but it was not used since the IPv6 specification already provides suggested values for the field, and these values require all 4 bits. Another option we considered was to define our own extension header and use the Next Header field to point to our header. The point of the Next Header field is, after all, to provide for extensions to the header; however, since the IPv6 header must be aligned on 4-byte boundaries, creating a new header for our two bits would have incurred an overhead of 30 extra padding bits to align the header, and we deemed this too wasteful. The final alternative we considered was to define two Next Header values, for example 128 and 64, which would give us two bits to use for ECN-Capable and ECN-Notify. However, this method would effectively eliminate 192 Next Header values, and it would grossly violate the spirit of the Next Header field. Thus, we decided to co-opt the two most significant bits of the Flow Label field for ourselves.

2.1. Router-side Support

An ECN router provides Explicit Congestion Notification to control congestion for ECN-capable transport protocols. Such a router uses a RED (Random Early Detection) queue management scheme to probabilistically mark packets and signal the sources about incipient congestion. However, while a RED router signals congestion by dropping these marked packets, an ECN router instead sets the ECN-Notify bit in the IPv6 header for ECN-capable flows. For non-ECN-capable flows, the behavior of an ECN router is the same as a RED router. The router determines if a packet is part of an ECN-capable flow by using the value of the ECN-Capable bit in the packet's header.

We now describe our design and implementation for adding ECN support to a router, and then we point out some of the benefits of our design choices.

2.1.1. Design Framework

We had several choices of design for implementing the ECN router. While ECN support could be written from scratch and added to the existing routing code, we chose instead to take advantage of the ALTQ (Alternate Queueing) framework [C97] designed and implemented by Sony Computer Labs, Japan for FreeBSD. Using this framework instead of implementing ECN support from scratch resulted in a number of benefits:

- the ALTQ framework is a relatively modular way of adding ECN support to a router running FreeBSD, which does not have the modules of a System V system;
- RED was already implemented for FreeBSD using the ALTQ framework, so ECN development could be done as an add-on to the already-tested RED code; and,
- by using this framework, all network drivers that also support the ALTQ framework could use ECN without any modification to their code.

In the following subsections we describe briefly how the alternate queueing framework is implemented, and then we describe how ECN fits into the framework.

2.1.1.1. Alternate Queueing Framework (ALTQ)

In the BSD kernel, queueing is performed at the network interface layer. The `ifnet` structure is an abstract network interface and is allocated for each network device. Since alternate queueing should be associated with the network interface, and most interface-related routines have a reference to the ifnet structure, alternate queueing fields were added to the ifnet data structure (see Figure 2).

```
struct ifnet
{
   /* ORIGINAL FIELDS HERE */
   int (*if_output)(....);   /* part of original field */

#ifdef ALTQ
   /* alternate queueing related stuff */
   int if_altqflags;         /*  altq flags (e.g., ready, in-use) */
   void *atlqp;              /*  altq state */

   int (*if_altqenqueue)(...);
   struct mbuf* (*if_altqdequeue)(..);
#endif
}
```

Figure 2: ALTQ-modified ifnet structure

The IP layer (both for IPv4 and IPv6) enqueues a packet by calling the `if_output()` routine associated with the interface. `ALTQ` changes the if_output routine associated with the ifnet structure to point to its own routines. Thus, alternate enqueueing and dequeueing operations can be implemented for any queue management scheme that supports `ALTQ`.

Various queue management schemes can be “attached” to and “detached” from any interface that supports ALTQ by manipulating the `if_altqenqueue()` and `if_altqdequeue()` function pointers that are a part of the modified `ifnet` structure. This manipulation is done by creating a new device and providing `ioctl`s for the device that attach and detach the device from the interface. In other words, a queueing scheme can be implemented by providing a device that implements that queueing scheme.

The main features of the ALTQ framework are

- flexibility to implement various queue management schemes (WFQ, CBQ, RED, ECN, etc.);
- modularity that isolates the code for the queue management schemes, enabling the same network driver to support multiple queueing mechanisms; and,
- easy-to-implement accounting information that can be used for network management.

2.1.1.2. ECN Router Implementation Using the ALTQ Framework

To provide ECN support for IPv6 using the existing `ALTQ` framework, we created a new device called "`ecn`." This device was added with a new minor device number of 5 for the existing major device "`altq`" with the major device number 88.

All queue management routines (`enqueue`, `dequeue`, etc.) that perform ECN were attached to the interface through ioctls for the `/dev/ecn` device. The following `ioctl`s are used to control the ECN device:

- **`ECN_ENABLE`** – enables ECN on a particular interface (enables alternate queueing)
- **`ECN_DISABLE`** – disables ECN on a particular interface (disables alternate queueing)
- **`ECN_IF_ATTACH`** – attaches ECN to a particular interface; any further packets sent to this interface will be handled by the ECN mechanism (note that `ECN_ENABLE` only enables alternate queue support to the interface; to add ECN support, we need to do `ECN_IF_ATTACH`)
- **`ECN_IF_DETACH`** – detaches ECN from a particular interface; packets sent to this interface are no longer handled by ECN
- **`ECN_GET_STATS`** – gets statistics, such as number of packets marked by ECN, number of packets dropped, average queue length, etc., for packets sent to this particular interface
- **`ECN_CONFIG`** – configures an ECN device, such as setting the queue length limit
- **`ECN_ACC_ENABLE`** – enables accounting for an interface
- **`ECN_ACC_DISABLE`** – disables accounting for an interface

Our ECN implementation for IPv6 is largely based on the RED implementation for IPv4 from [FJ93]. Currently the same parameters for queue management from this RED implementation were used in our ECN implementation. These values are

- minimum queue size threshold = 5 packets;
- maximum queue size threshold = 15 packets;
- maximum queue size = 60 packets; and,
- Q_weight (of RED algorithm) = 0.0095 (= 511/512).

2.1.2 Features of ECN Router Implementation

Our ECN router implementation has a number of nice features. First, for non-ECN-capable packets, the router uses the RED queue management scheme and "signals" congestion by dropping packets. Thus, the router supports both ECN-capable and non-ECN-capable traffic; this means it can be deployed gradually, and such deployment would be transparent to hosts. Our implementation also supports IPv4, IPv6, and IPv6 tunnels, since it uses the ALTQ framework. (Although IPv6 tunnels are supported, we have not tested this feature.) Since ALTQ provided an implementation of RED, we inherited its mechanisms for free. And finally, our implementation supports any ALTQ-enabled network drivers without modification.

2.2. Host-side Support

Hosts broadly require three new features to support ECN: negotiating to use ECN at connection setup time, having the receiver relay back to the sender that congestion was encountered in the sender's packet, and having the sender react appropriately to such notifications from the receiver.

To negotiate use of ECN, the initiator of the TCP connection (the client) is responsible for offering to use ECN. The client turns on the ECN-Capable bit to signal its willingness to use ECN. If the server is also willing to do ECN, it will also turn on the ECN-Capable bit in its acknowledgment of the client's initial SYN packet. From that point on, for the life of the connection, ECN will be used. If either the client or the server won't do ECN, then ECN will not be used for that connection and both sides are free to ignore the ECN bits. Note that in our negotiation protocol, the use of ECN applies in a full-duplex sense. Potentially, if the connection was highly asymmetric with respect to congestion, each direction of the connection might want to independently negotiate ECN, but this is left for future research.

We implemented the negotiation by creating two new states for TCP connections: **`TCPS_SYN_RECEIVED_ECN`** and **`TCPS_ESTABLISHED_ECN`**. These states represent a client who has requested a new connection using ECN and an established connection which uses ECN, respectively. `tcp6_output()` was then modified to turn on the ECN-Capable bit when the connection was in the appropriate state (see Figure 3). Note that ECN-Capable is not turned on for packets that are pure acknowledgments with no data, since there is no way to handle this packet if it were ECN-marked.

```
#ifdef ECN
  /*
   * Turn on the ECN-Capable bit in three cases:
   * 1. negotiating to use ECN when setting up  cxn (TCPS_SYN_SENT)
   * 2. agreeing to client's offer to use ECN (TCPS_SYN_RECEIVED_ECN)
   * 3. already agreed to use ECN for this  cxn (TCPS_ESTABLISHED_ECN),
   *    but not if the packet is a pure ACK with no data
   */
  if ((tp->t_state == TCPS_SYN_SENT) ||
      (tp->t_state == TCPS_SYN_RECEIVED_ECN) ||
      ((tp->t_state == TCPS_ESTABLISHED_ECN) && ! pureack))
  {
#ifdef ECN_DEBUG
    if (tp->t_state == TCPS_SYN_SENT)
      printf ("tcp6_output: negotiating to use  ECN\n");
    else if (tp->t_state == TCPS_SYN_RECEIVED_ECN)
      printf ("tcp6_output: agreeing to client's ECN  negotiation\n");
#endif
    IPV6_ECN_CAPABLE_SET ( ti->ti6_i);
  }
#endif
```

Figure 3: Setting the ECN-Capable bit

In an ECN connection, when a host receives data with the ECN-Notify bit on, it must somehow inform the sender that congestion has occurred. At first we might simply decide to turn on the ECN-Notify bit in the ACK for that data; however, as mentioned above, use of ECN applies in both directions, and it is common to piggyback an ACK onto a reverse-direction data packet. To turn on the ECN-Notify bit on this data packet would imply to the sender that this piggybacked ACK itself encountered congestion on its way back to the sender. Instead, we need to convey the congestion

message without using the ECN-Notify bit ourselves. To this end, we have chosen one of the "reserved" bits of the TCP header to signal this condition in our ACKs. The reserved bit we have chosen is the least significant bit of the "Reserved" field, leading to our own revised TCP header (see Figure 4). **ENA** stands for ECN Notify Acknowledgment.

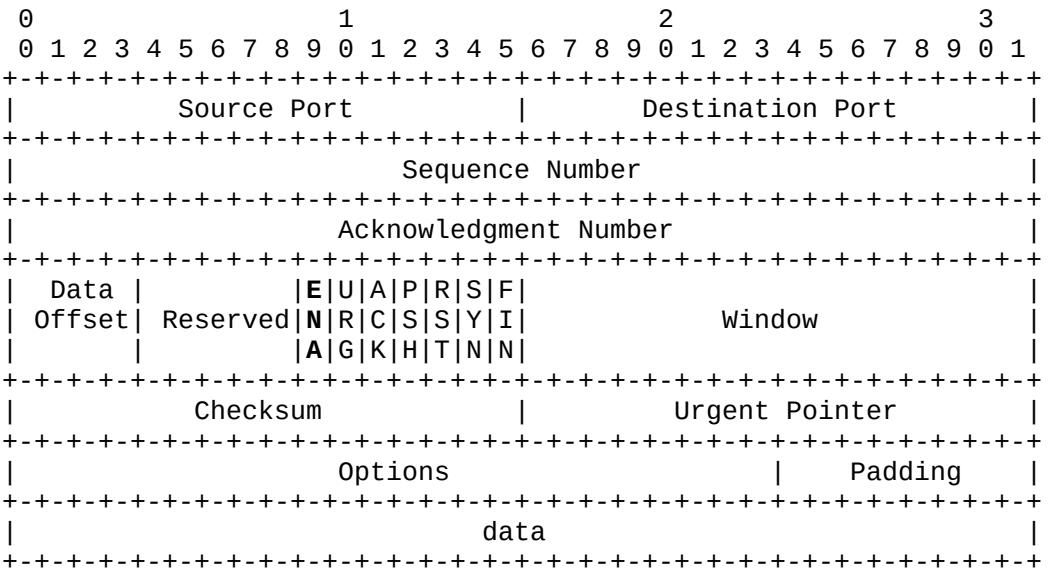

Figure 4: Modified TCP header format

We implemented this ECN Notify Acknowledgment by creating a new flag called **TF_ECN_ACKNOTIFY** in the `tcpcb` (TCP control block). This flag corresponds to the desire to set the ENA bit in outgoing packets; it is turned on by `tcp6_input()` when three conditions are all true:

- the state of this TCP connection is `TCPS_ESTABLISHED_ECN`;
- the `EN` bit in the IP header is on for the incoming packet; and,
- this host, as the receiver of this packet, is acknowledging the packet.

When TF_ECN_ACKNOTIFY is on, the ENA bit is turned on via the **TH_ECN_NOTIFY** mask. This mask corresponds directly with the ENA bit in the outgoing TCP packet's header.

The third feature required on the host to support ECN is having the sender react if the ENA bit is set. The sender should react in a manner similar to the reaction to having a packet dropped – it should reduce its `cwnd` and `ssthresh` by half. However, there is additionally one very important detail to implement: hosts should not react to congestion notifications more than once per window of packets sent. In this sense, "congestion notification" can be any of ECN-marked packets, detection of dropped packets, or duplicate acknowledgments. The meaning behind this requirement stems from the fact that when congestion occurs at a router, it is likely that multiple packets will be marked to signal congestion. However, reacting to congestion for each one of these marked packets which are within the same window is far too conservative. Thus, the effects of small bursts of congestion are smoothed by this requirement.

This feature required extensive modifications to the TCP code. To implement the sender's reaction, `tcp6_input()` was modified to add a check for the `ENA` bit when ACKs are processed. This happens in two place in that function: during header prediction, and in the normal processing of the segment (step 7 of the finite state machine). The processing code was adapted from the handling of the retransmit timer going off. Secondly, checks needed to be added to prevent multi-

ple congestion reactions within one window. This involved adding a new variable called **`snd_ignore_cong`** to the TCP state structure (`struct tcpcb`). This variable is set to `snd_next` when the ENA bit is detected; thus, it remembers the what packets were "in flight" when a marked packet is acknowledged. When it is set, congestion control due to the retransmit timer, duplicate ACKs which lead to a fast retransmit, or another ENA will not occur.

3. Testing and Verification

We used three Intel-based computers installed with FreeBSD 2.2.2 [FBD97]: `gridlock`, `bottleneck`, and `sigalert`. The machines were arranged so that bottleneck acted as a router to the other two machines (see Figure 5). Point-to-point 10BaseT links connected both gridlock and sigalert to one of two identical Ethernet cards in bottleneck. Bottleneck was additionally connected to the UCLA Department of Computer Science network using a third 10BaseT card. This meant that all network traffic to sigalert and gridlock was routed through bottleneck.

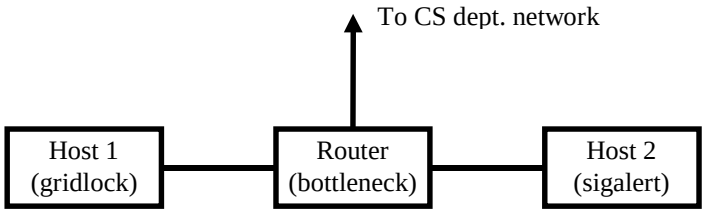

Figure 5: Network configuration

We used tg, a traffic generator program, for generating the TCP traffic at the rates we wanted. tg also generates statistics about the effective throughput of its test runs. To monitor the state of our TCP connections, specifically the current congestion window size (`cwnd`) and the slow-start threshold (`ssthresh`), logging messages were inserted in the TCP kernel code to log these values.

Tests were run by sending traffic from bottleneck to sigalert in the desired configuration, described below. Originally, we had planned to send traffic from gridlock to sigalert, but in the last two weeks of this project gridlock began experiencing spontaneous kernel panics and reboots. These occurrences were linked to network activity, and thus we didn't trust the validity of any network tests run from gridlock. We finally opted to omit gridlock altogether from our testing configuration.

We tested the behavior of both the router and the end-to-end performance separately. First, we tested that the router was managing the queue lengths appropriately. Then we tested for the effective throughput of the entire system. Our tests each had durations of 120 seconds. We ran each test both with and without ECN, and each we tested each case three times and averaged the results. (Note that without ECN, FreeBSD's routing uses the drop-tail algorithm.) The values we measured from each test were the effective throughput, from the tg logs, and the percentage of marked packets (using ECN only) and percentage of dropped packets, from our kernel logging code. Both the sender and receiver buffers were set to the maximum (65,535 bytes) to force the limiting factor for our TCP windows to be the cwnd value.

Great care was taken to avoid other activity, both network and system, on the two machines while running our tests. This included NFS traffic (we used bottleneck as our NFS server for our home directories), active telnet sessions to CS department servers, and even the load of running Xwindows on these systems.

Each end-to-end test we ran was designed to measure a specific aspect of ECN's performance. The aspects we measured were performance with no congestion, performance with congestion, fairness, and throughput improvement.

3.1 Test 1: Demonstration of the router-side implementation

The first test was performed to demonstrate how ECN controls average queue length in the router. The packets were marked under various traffic conditions, as summarized in Figure 6. We sent the flows as described in the following cases:

A. 1 flow of 5.46 Mbps (shown in Figure 7)
B. 3 flows of 10 Mbps each (shown in Figure 8)
C. 6 flows of 10 Mbps each (shown in Figure 9)

3.2. Test 2: Performance with no congestion

A single TCP flow sent 4096-byte packets at 6 msec intervals, for a bandwidth of 5.46 Mbps, much less than the available 10 Mbps in the link.

3.3. Test 3: Performance with congestion

A single TCP flow sent 8192-byte packets at 6 msec intervals, for a desired bandwidth of 10.92 Mbps. As this is larger than the available bandwidth, congestion should occur.

3.4. Tests 4 and 5: Fairness

Three tests were run to evaluate fairness. By fairness, we mean that flows that send more traffic have more packets parked. In test 3, three identical TCP flows sent fixed-size packets at a desired rate of 10.92 Mbps each. And in test 4, three TCP flows sent Poisson-distribution-sized packets with desired rates of 1.37 Mbps, 5.46 Mbps, and 9.56 Mbps, respectively.

3.5. All end-to-end tests (2 – 5): Throughput improvement

No specific test evaluated throughput improvement. Instead, the aforementioned end-to-end tests were used to measure the throughput improvement from using the default congestion control to using ECN.

4. Testing Results

4.2. Router test results

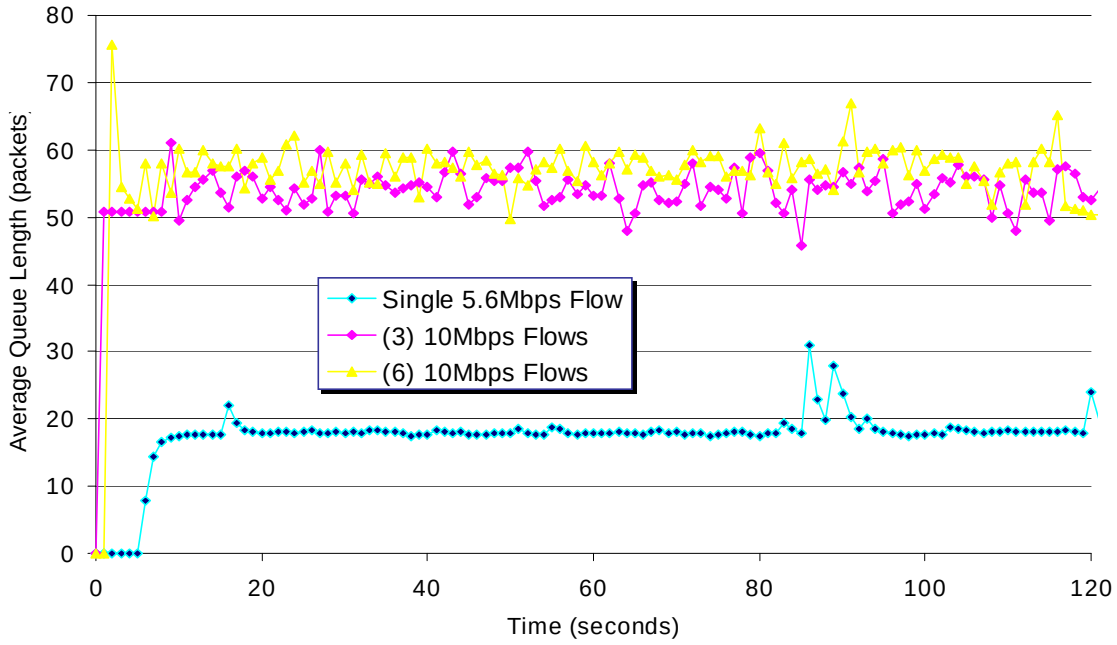

Figure 6: Summary of router tests (all cases)

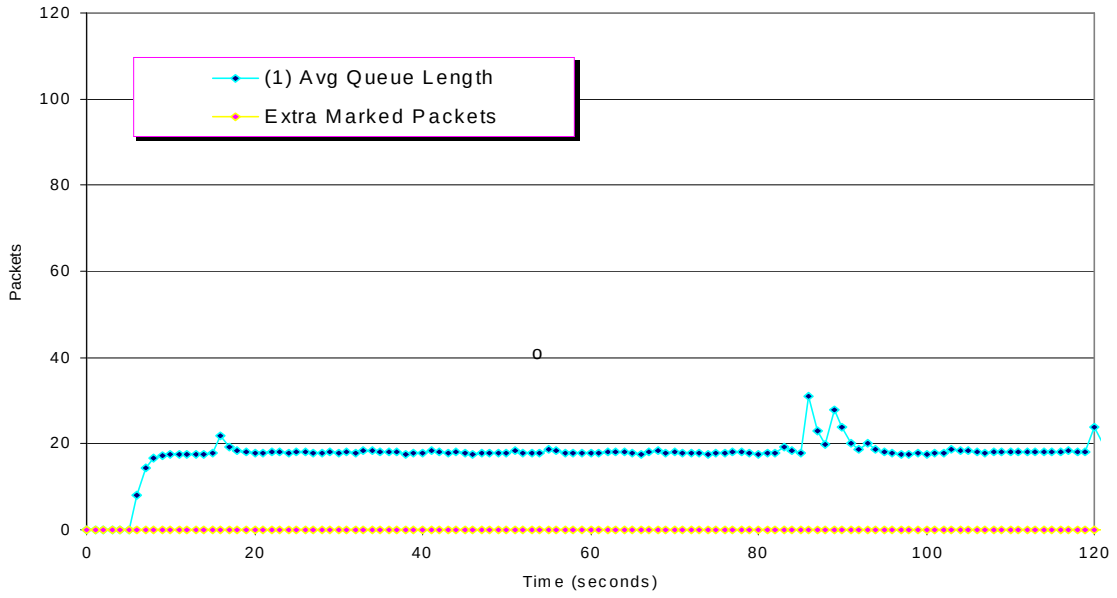

Figure 7: Router test case A: uncongested router (1 flow of 5.46 Mbps traffic)

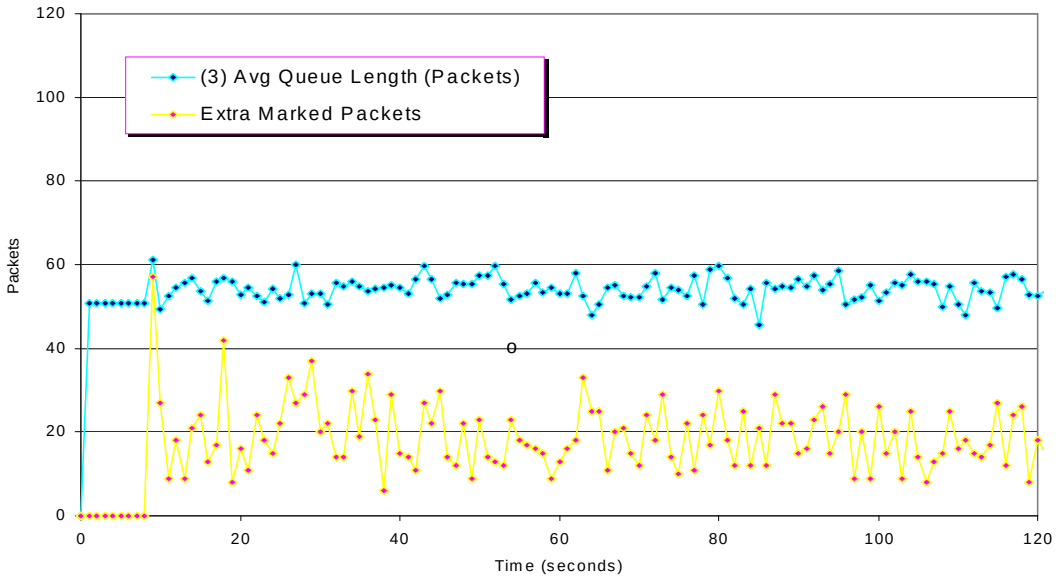

Figure 8: Router test case B (congested router, 3 flows of 10.49 Mbps traffic)

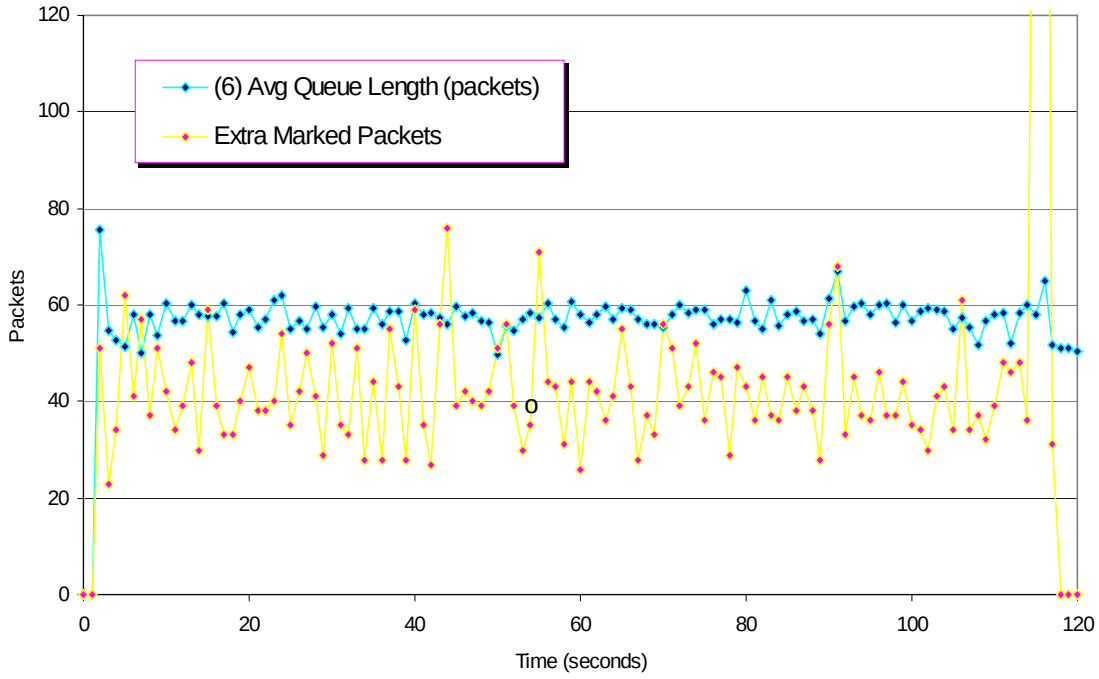

Figure 9: Router test case C (congested router, 6 flows of 10.49 Mbps traffic)

4.2. Results of end-to-end tests (tests 2-5)

Test	Offered traffic	ECN?	Mean through-put	Mean marked packets	Mean dropped packets	Through-put improve-ment by ECN
2 (Perf. w/o cong.)	5.46Mbps	No	5.46Mbps	N/A	0.0%	0.00%
		Yes	5.46Mbps	0.00%	0.0%	
3 (Perf. w/ cong.)	10.92Mbps	No	6.89Mbps	N/A	0.01%	1.60%
		Yes	7.00Mbps	15.35%	0.00%	
4 (Fairness, equal flows)	10.92Mbps x 3 flows	No	2.26Mbps, 2.20Mbps, 2.14Mbps (2.20Mbps average)	N/A	23.50%, 22.18%, 24.01% (23.23% average)	4.55% average
		Yes	2.30Mbps, 2.24Mbps, 2.35Mbps (2.30Mbps average)	67.04%, 65.12%, 64.20% (65.45% average)	0.0%, 0.83%, 0.0% (0.28% average)	
5 (Fairness, unequal Poisson flows)	1.37Mbps, 5.46Mbps, 9.56Mbps (16.39 Mbps to-tal)	No	1.54Mbps, 2.64Mbps, 2.58Mbps (6.76Mbps total)	N/A	1.32%, 2.61%, 6.51%	1.78% total
		Yes	1.54Mbps, 2.60Mbps, 2.74Mbps (6.88Mbps total)	16.95%, 17.53%, 28.00%	0.17%, 0.00%, 0.00%	

5. Analysis

5.1. Test 1: Demonstration of the router-side implementation

The results of case A, shown in Figure 7, proves that ECN marked packets only when congestion occurred. Since the queue length was lower than minimum threshold of congestion, we can see that ECN did not mark even a single packet.

The results of cases B and C, as shown in Figure 8 and Figure 9, show how our ECN implementation controlled the queue size well. This was due to the RED queue management scheme and notifying senders of congestion by marking packets. We can see that more packets were marked for higher traffic rate sources. We can also see that the queue length was larger than the minimum threshold, which caused ECN to mark packets.

The summary plot shown in Figure 6 shows that average queue length was higher for higher traffic rates and smaller for lower traffic rates. Note that ECN tried to control the average queue size even for higher traffic rates.

In all the plots in Figures 6 to 9, we can see the randomization effect that was due to the RED queue management scheme.

5.2. Test 2: Performance with no congestion

As expected, with no congestion, ECN did not affect the throughput at all. This should come as no surprise, as there were no packet marks or drops for such a light load.

5.3. Test 3: Performance with congestion

With this amount of congestion, ECN marked many more packets than the drop-tail router dropped packets; however, this still resulted in ECN better throughput. This supports our earlier assertion that dropping a packet degrades performance much more severely than marking a packet.

5.4. Tests 4 and 5: Fairness

In test 3, we note that the three flows are sending at the same rate, and the percentages of marked packets are all also nearly the same. Additionally, test 4 shows that the number of marked packets corresponds with the offered traffic for each flow (just as the dropped packets corresponds as well for the non-ECN case). These test results thus demonstrate the fairness of ECN.

5.5. All end-to-end tests: Throughput improvement

In all end-to-end tests but test 2, the throughput using ECN was larger than the throughput without ECN. This improvement was not as dramatic as one might expect; however, simulations of ECN by Floyd [F94] also showed that throughput improvement is generally small.

6. Conclusion

ECN seems to help in cases of severe congestion, but its improvement in throughput is marginal. This can be explained by comparing the effect of a single marked packet versus a single dropped packet. If a packet is marked, the cwnd and ssthresh will be halved, whereas if a packet is dropped, ssthresh will still be halved but cwnd will be reduced to 1 MTU. In the next few round-trip times, in the case of a drop, the flow's cwnd will have grown exponentially (since it is less than ssthresh), but in the case of a mark, the flow's cwnd will only have linearly grown (since it is equal to or greater than ssthresh, if the receiver's advertised window is sufficiently large). Thus within a few round-trip times, the cwnd for the flow that experienced the drop will have nearly caught up with the cwnd for the flow that experienced the mark. This may explain the low increase in throughput.

A more dramatic measure of ECN's improvement is the packet delay [F94]. We unfortunately did not explicitly test for this result due to time constraints. The results of our router testing, however, do tend to imply lower packet delays using ECN due to the excellent maintenance of its queue length.

7. Project surprises

"Life is like a box of chocolates."

So is our project. The following is a list of things we did not expect initially.

- Installing and configuring FreeBSD and especially IPv6 took much longer time than we had expected. This led to some schedule slippage.
- We thought that the TCP receiver would need to notify the TCP sender of congestion with a TCP option. Later we realized that we can just use one of the unused "reserved" bits in the TCP header. This greatly reduced our programming effort, the overhead, and the complexity of the ECN implementation on the host side.
- We thought that we would need to implement our own RED algorithms on the router, but later we found ALTQ from Sony Computer Science Laboratory. This greatly reduced our programming effort on the router side.
- Gridlock's failure occurred at a very inopportune time. We even reinstalled FreeBSD, but it still generated repeated kernel panics, so we gave up on using it. We thus have no idea how much our tests are affected by both the sending and routing being on the same machine.
- Using logging messages in the kernel to monitor cwnd/ssthresh values is very crude. The logging is lossy, especially when outputting the volume of data generated by our testing, and so we are certain that some amount of log data was lost in the process. However, we were unable to think of a better way to continuously display cwnd and ssthresh values for specific flows.
- Creating the right amount of congestion was more difficult than we'd originally thought. We spent many long hours trying to create enough congestion to demonstrate ECN's benefits but not so much congestion that would total fill the router's queues (which would lead to all packets being dropped, regardless).
- And finally, when using a word processor to create the final report, make sure everyone has the same version!

8. Future Research

⇒ As mentioned earlier, our negotiation forces use of ECN in both directions. An open question is, would anyone ever want to use it in only one direction?

⇒ TCP is the only transport-layer protocol for which ECN is implemented. Implementing ECN for other protocols of the transport and higher layers is subject to future research.

⇒ Our testing methods left a lot to be desired. A more thorough testing would definitely be a step in the right direction to more properly verify our implementation. Such an effort should include determining a better way to monitor TCP's cwnd/ssthresh values.

⇒ Additionally, different network topologies should be tested. Three such possibilities are alternate LAN configurations, WANs or other networks with longer round-trip times, and IPv6 tunnels, which should be supported in our implementation but was not tested.

9. Acknowledgments

We are greatly indebted to the following people who generously helped us:

- Andreas Terzis, for helping install and troubleshoot FreeBSD on our test machines.
- Yixin Jin, for helping install and troubleshoot IPv6 on our test machines.
- Scott Michel, for providing us a copy of his RED implementation for Solaris.

10. References

[C97] Cho, K. *Alternate Queueing for BSD Unix*, Sony Computer Science Laboratory, February 1997.

[DH95] Deering, S., and Hinden, R. *Internet Protocol, Version 6 (IPv6) Specification*, RFC 1883, December 1995.

[F94] Floyd, S. *TCP and Explicit Congestion Notification*, ACM Computer Communication Review, V. 24 N, October 1994.

[FJ93] Floyd, S., and Jacobson, V. *Random Early Detection Gateways for Congestion Avoidance*, IEEE/ACM Transactions On Networking, August 1993.

[FJ94] Floyd, S., and Jacobson, V. *The Synchronization of Periodic Routing Messages*, IEEE/ACM Transactions on Networking, April 1994.

[FBD97] FreeBSD Board of Directors, *FreeBSD Handbook*, URL <http://www.freebsd.org>, 1997.

[J88] Jacobson, V. *Congestion Avoidance and Control*, Proceedings of SIGCOMM 1998, August 1988.

[P81] Postel, J. (editor). *Transmission Control Protocol DARPA Internet Program Protocol Specification*, RFC 793, September 1981.

[RF97] Ramakrishnan, K. K., and Floyd, S. *A Proposal to add Explicit Congestion Notification (ECN) to IPv6 and to TCP*, IETF Draft draft-kksjf-ECN-00.txt, November 1997.